

Bare-Metal Mikrodenetleyicilerde FastGRNN: Gerçek Zamanlı İnsan Aktivite Tanıma için Düşük Ranklı, Seyrek ve Nicelendirilmiş Çıkarımın Uçtan Uca Yeniden Üretimi

Emre Can Kızılateş
Bağımsız Araştırmacı, İzmir, Türkiye
kizilatesemrecan@gmail.com

Özet—Son yıllarda makine öğrenmesindeki hâkim eğilim, zor problemleri daha büyük modeller, daha güçlü hızlandırıcılar ve daha geniş bellek bütçeleriyle çözmek oldu. Ancak yıllardır süren küresel yarı iletken arz sıkıntısı [1], [2] ve sürekli çevrim içi çalışan çıkarım sistemlerinin artan enerji ve karbon maliyeti [3], [4], bu yaklaşımın kırılabilirliğini açık biçimde ortaya koymaktadır. Bu tablo, ters yönde bir arayışı da gerekli kılar: yapay zekâ ve makine öğrenmesi algoritmalarını, giyilebilir cihazlarda, sensörlerde ve uç sistemlerde zaten yaygın biçimde kullanılan küçük mikrodenetleyicilere sığacak biçimde yeniden tasarlamak. Bu çalışma bu arayışa odaklanmaktadır. Kaynak kısıtlı çıkarım için geliştirilmiş kompakt bir kapılı yinelemeli hücre olan FastGRNN [5] algoritmasını uçtan uca ve açık kaynaklı olarak yeniden üretiyor; modeli, mevcut silikon ekosisteminin alt ucunu temsil eden iki bare-metal hedefte çalıştırıyoruz: 8-bit Arduino Uno R3 (ATmega328P) ve 16-bit MSP430G2553 (donanım çarpıcısı yok; 16 KB Flash; 512 B SRAM). Sıkıştırma boru hattı düşük ranklı ağırlık ayrıştırmasını, iteratif sert eşikleme ile seyreklik kazandırmayı ve açık aktivasyon kalibrasyonu içeren tensör-başına Q15 eğitim-sonrası nicelendirmeyi bir araya getirir. Ortaya çıkan modelin ağırlıkları yalnızca 566 bayt yer kaplar; model HAPT kıyaslama kümesinde makro $F1 = 0.918$ değerine ulaşır ve 3,399 test penceresinin tamamında PyTorch referansıyla %100 tahmin uyumu gösterir. Her iki platform da 50 Hz’de gerçek zamanlı akış halinde çıkarım yapabilir (Arduino’da örnek başına 9.21 ms; MSP430’da 13 ms). Ayrıca 256 girişli sigmoid/tanh arama tablosu, donanım çarpıcısı bulunmayan MSP430 üzerinde $30.5\times$ hızlanma sağlar. Çalışma özgün FastGRNN makalesini üç noktada genişletir: (i) platformlar arasında bit-eşdeğer deterministik çıkarım; (ii) uçtan uca yanıt süresini belirleyen yaklaşık 2 saniyelik yinelemeli ısınma gecikmesinin nicel incelenmesi; ve (iii) çarpıcısız gömülü hedeflerde uygulanabilir bir arama tablosu yöntemi. Sonuçlar, kompakt yinelemeli mimarilerin kalibre edilmiş nicelendirme ve arama tablosu tabanlı aktivasyonlarla birlikte, özel hızlandırıcılara gerek duymadan kaynak kısıtlı mikrodenetleyicilerde gerçek zamanlı cihaz-üstü çıkarımı mümkün kılabilirliğini göstermektedir.

Index Terms—FastGRNN, yinelemeli sinir ağları, insan aktivite tanıma, nicelendirme, seyreklik, düşük ranklı ayrıştırma, uç yapay zekâ, tinyML, mikrodenetleyici, MSP430, Arduino, bare-metal çıkarım.

I. GİRİŞ

A. Motivasyon: Silikon Büyümediginde Modeli Küçültmek

Makine öğrenmesinde son on yıla damgasını vuran yaklaşım oldukça belirgindir: problem zorlaştıkça model büyütülür, daha

güçlü bir hızlandırıcı kullanılır ve bellek bütçesi genişletilir. Bu strateji dil, görüntü ve çok kipli üretim alanlarında etkileyici sonuçlar vermiştir [4]. Bununla birlikte aynı stratejinin enerji, karbon ve tedarik zinciri maliyeti de giderek daha görünür hâle gelmiştir. 2020–2024 döneminde ileri seviye yarı iletkenlerde yaşanan küresel arz sıkıntısı otomotiv, tüketici elektroniği ve tıbbi cihaz sektörlerini aksatmış [1], [2]; sürekli çevrim içi çalışan çıkarım sistemlerinin enerji tüketimi ise hem büyük ölçekli hem de hızla büyüyen bir yük hâline gelmiştir [3], [4].

Bu nedenle tamamlayıcı bir araştırma yönü giderek daha önemli hâle gelmektedir. Buradaki soru, bir modeli çalıştırmak için hangi yeni hızlandırıcıya ihtiyaç duyduğumuz değil; hâlihazırda kitlesel ölçekte üretilen, her yıl on milyarlarca adet sevk edilen ve birim maliyeti birkaç dolar düzeyinde kalan silikon üzerinde hangi modellerin çalışabileceğidir. Bu silikon; giyilebilir cihazlarda, sensörlerde, akıllı ev aygıtlarında, otomotiv alt sistemlerinde ve endüstriyel telemetri uç noktalarında kullanılan 8-bit ve 16-bit mikrodenetleyicilerdir (MCU). Bu alan genellikle “tinyML” olarak anılır [6]; GPU ölçekli çıkarımdan farklı olarak belirli bir büyük ölçekli altyapıya bağlı değildir.

B. Bare-Metal MCU Sınıfı

Yayımlanmış tinyML uygulamalarının önemli bir bölümü ARM Cortex-M sınıfı hedeflere yönelir. Bu aygıtlar yüzlerce kilobayt SRAM, donanım kayan nokta birimleri, tek çevrimli çarpıcılar ve üretici tarafından iyileştirilmiş sinir ağı çekirdekleri sunar [7]–[9]. Ucuzdurlar; ancak mikrodenetleyici dünyasının en alt ucuna göre hâlâ oldukça güçlüdürler. Bu makalede incelediğimiz *bare-metal* MCU’lar ise kullanılabilir silikon yelpazesinin öteki ucunda yer alır:

- 32 KB Flash, 2 KB SRAM, donanım 8×8 çarpıcısı ve kayan nokta birimi bulunmayan 8-bit Arduino Uno R3 (ATmega328P);
- 16 KB Flash, 512 B SRAM ve hiçbir tür donanım çarpıcısı bulunmayan 16-bit MSP430G2553. Bu aygıtta çarpma işlemleri yazılımla öykünülür.

Bu iki parça bugün de aktif olarak üretilmektedir ve birim maliyetleri ARM Cortex-M sınıfı hedeflerin belirgin biçimde

altındadır. Bir yinelemeli ağ MSP430G2553 üzerinde gerçek zamanlı çalıştırılabilirse, aynı ağın bugün üretimde olan neredeyse her yaygın mikrodenetleyiciye uyarlanabileceğini söylemek mümkündür.

C. Neden Yinelemeli Ağlar, Neden FastGRNN?

Bileğe ya da bele takılan bir eylemsiz ölçüm sensöründen insan aktivite tanıma (HAR), akış halinde sınıflandırma için doğal bir örnektir. Giriş boyutu düşüktür (üç ivmeölçer kanalı), örnekleme hızı sınırlıdır (50 Hz) ve çıkış uzayı küçüktür (altı sınıf) [10], [11]. Bu nedenle problem, bare-metal MCU sınıfını değerlendirmek için uygundur: modele sığacak kadar küçük, fakat pencerelenmiş ileri beslemeli bir taban çizgisinden yarar görece kadar zamansaldır.

FastGRNN [5], kaynak kısıtlı çıkarım için tasarlanmış kapılı bir yinelemeli hücredir. Düşük ranklı ağırlık ayrıştırması, iteratif sert eşikleme (IHT) ile seyrekleştirme [12] ve Q15 sabit noktalı nicelendirme [13], [14] gibi birbirini tamamlayan sıkıştırma tekniklerini bir araya getirir. Özgün çalışmada FastGRNN'nun, LSTM [15] doğruluğuna çok daha az parametreyle yaklaşabildiği gösterilmiştir. Ancak bu çalışma çarpıcısız bir hedef için herkese açık bare-metal bir başvuru uygulaması sunmaz; ayrıca modelin akış kipindeki davranışını ayrıntılı biçimde incelemeyiz.

D. Katkılar

Bu makale, bare-metal MCU'larda HAR için FastGRNN algoritmasının uçtan uca ve açık kaynaklı bir yeniden üretimini sunar. Özgün makalenin ötesinde üç katkı öne çıkmaktadır:

- **Platformlar arası deterministik çıkarım.** Tek bir taşınabilir C kaynak dosyası hem 8-bit Arduino Uno R3 hem de 16-bit MSP430G2553 üzerinde derlenir. 3,399 test penceresinin tamamında *bit-eşdeğer* gizli durum yürüngeleri üretir ve PyTorch referansı ile tahmin düzeyinde %100 uyur. Donanımdan bağımsız bu düzeyde yeniden üretilebilirlik, güvenlikle ilişkili gömülü uygulamalar için özellikle değerlidir.
- **Çarpıcısız hedefler için uygulanabilir bir arama tablosu yöntemi.** $[-8, +8]$ giriş aralığında tanımlanan 256 girişli sigmoid/tanh arama tablosu, MSP430G2553 üzerinde tam pencere çıkarımını $30.5\times$ hızlandırır (54 s $\rightarrow$ 1.8 s). Böylece gerçek zamanlı çalışamayan bir uygulama, 50 Hz akış bütçesini rahatlıkla karşılayan bir hedefe dönüşür. Bu yöntem FastGRNN'ya özgü değildir; σ ya da tanh aktivasyonları kullanan başka yinelemeli hücrelere de uygulanabilir.
- **Yinelemeli ısınma gecikmesinin ölçülmesi.** Tahminlerin kararlı hâle gelmesi için, donanımdan bağımsız olarak, yaklaşık 2 saniyelik (~ 100 örnek, 50 Hz) gizli durum geçmişini gerektiğini gözlemliyoruz. Bu davranış donanımdan değil, yinelemeli dinamiklerden kaynaklanır ve FastGRNN sınıfı HAR sistemlerinde kullanıcıya yansıyan yanıt süresini belirler. Özgün makalede bu nokta ele alınmamıştır.

Kod ve yeniden üretilebilirlik. Kaynak kodu, eğitilmiş modeller, dışa aktarılan Q15 ağırlıkları, deney günlükleri ve gö-

mülü hedefler için üretilen ikililer Apache Lisansı 2.0 altında <https://github.com/emre1998/fastgrnn-har> adresinde paylaşılmıştır. Tam yeniden üretim yaklaşık iki saat CPU eğitimi ve beş dakika MCU aktarımı gerektirir.

E. Makalenin Akışı

Bölüm II, kompakt RNN ve uç-ML literatürünü özetler. Bölüm III, FastGRNN hücresini ve düşük ranklı, seyrek, niceleştirilmiş (L-S-Q) sıkıştırma boru hattını tanımlar. Bölüm IV deneysel kurulumu açıklar. Bölüm V doğruluk, bellek ayak izi ve gerçek zamanlı akış performansını raporlar. Bölüm VI, ısınma gecikmesini ve platformlar arası determinizmi tartışır; ayrıca sınırlılıkları verir. Bölüm VII makaleyi sonuçlandırır.

II. İLGİLİ ÇALIŞMALAR

Bu çalışma, beş ayrı fakat birbiriyle yakından ilişkili literatür çizgisiyle bağlantılıdır.

A. Kompakt Yinelemeli Hücreler

Standart LSTM [15] ve GRU [16] hücreleri uzun süreli zamansal örüntüleri etkili biçimde yakalar; ancak kilobayt düzeyinde belleğe sahip MCU'lar için ağır kalır. Örneğin orta büyüklükte bir LSTM ($H=64$, $d=3$) FP32 duyarlılıkta daha şimdiden 50 kB üzerinde ağırlık gerektirir. Bu değer, hedef aygıtımızın 16 kB Flash sınırını aşar.

FastGRNN [5], bu boşluğu düşük ranklı ağırlık matrisleriyle desteklenen iki skaler kapılı (ζ, ν) bir hücreyle kapatmayı amaçlar. Dizi görevlerinde LSTM ile karşılaştırılabilir doğruluğa, bir ila iki büyüklük mertebesi daha az parametreyle ulaşabilir. Yinelemeli olmayan tamamlayıcı EdgeML yaklaşımları arasında sığ doğrusal olmayan bir ağaç olan Bonsai [17] ve prototip tabanlı bir sınıflandırıcı olan ProtoNN [18] yer alır. Bu üç yöntem Microsoft Research EdgeML araç takımını oluşturur [19]. HAR için kritik olan zamansal girdiyi, bu üçlü içinde doğrudan işleyen yöntem FastGRNN'dur.

B. Nicelendirme

Eğitim-sonrası nicelendirme (PTQ), sinir ağı ağırlıklarını ve aktivasyonlarını FP32 yerine sabit noktalı ya da düşük bitli tamsayı biçimlerinde temsil eder. Jacob *vd.* [13], tensör-başına ölçek kullanan ve yalnızca tamsayı aritmetiğine dayanan çıkarımı tanıtmıştır. Han *vd.* [14] ise budama, nicelendirme ve Huffman kodlamasını birleştirerek derin ağları büyük ölçüde sıkıştırmıştır. Nicelendirme farkında eğitim (QAT) [20], PTQ sonrasında görülebilen doğruluk kaybını azaltır; ancak bunun bedeli yeniden eğitimidir. Bu çalışmada hem ağırlıklar hem de, açık aktivasyon kalibrasyonu ile, ara tensörler için tensör-başına Q15 PTQ kullanıyoruz. Kalibrasyon adımı pratikte kritiktir: Bölüm V-D'de gösterildiği gibi, kalibrasyon yapılmadığında model çöker; kalibrasyon yapıldığında ise QAT'ye gerek kalmadan doğruluk korunur.

C. Seyreklik ve Budama

FastGRNN'da kullanılan seyrekleştirme yöntemi iteratif sert eşiklemedir (IHT) [12]. Her eğitim adımında, her ağırlık tensöründeki mutlak değerce en büyük k girdi korunur; diğer girdiler sıfırlanır ve ağ bu sabit maske ile eğitime devam

eder. Han *vd.* [21], daha büyük ağlarda agresif budamanın doğruluğu koruyabildiğini göstermiştir. Frankle ve Carbin'in piyango bileti hipotezi [22] de bu tür seyrek alt ağların neden eğitilebilir kaldığına kuramsal bir zemin sağlar.

D. Daha Güçlü Hedeflerde tinyML

tinyML literatürü [6] çoğunlukla ARM Cortex-M sınıfı hedeflere odaklanır. Bu hedeflerde yüzlerce kilobayt SRAM, donanım FPU'ları ve tek çevrimli çarpıcılar bulunur. MCU-Net [9], Cortex-M verimini artırmak için sinir mimarisini ve yorumlayıcıyı birlikte tasarlar. CMix-NN [23], bellek kısıtlı uç aygıtlar için karma duyarlılık çekirdekler sunar. ARM'ın CMSIS-NN kütüphanesi [7], yaygın katmanları Cortex-M SIMD yönergeleriyle hızlandırır. TensorFlow Lite Micro [8] ise taşınabilir bir çalışma zamanı katmanı sağlar.

Bu çalışmaların hiçbirisi, bu makalede ele alınan MSP430G2553 sınıfı aygıtı hedeflemez. Bu aygıt bir ila iki büyüklük mertebesi daha küçüktür ve donanım çarpıcısına sahip değildir. Bildiğimiz kadarıyla bu çalışma, donanım çarpıcısı bulunmayan bir MCU üzerinde kapılı yinelemeli bir hücrenin herkese açık ilk uçtan uca yeniden üretimidir.

E. İnsan Aktivite Tanıma

UCI HAR [10] ve geçiş etiketlerini de içeren uzantısı HAPT [11], ivmeölçer tabanlı HAR için yaygın kullanılan kamusal kıyaslama kümeleridir. DeepConvLSTM [24], yığılmış CNN ve LSTM katmanlarıyla güçlü bir derin öğrenme taban çizgisi oluşturmuştur. Wang *vd.* [25] ile Demrozi *vd.* [26] tarafından hazırlanan taramalar, yönetsel manzarayı geniş biçimde özetler. Her iki tarama da dinamik DOWNSTAIRS sınıfının kalıcı bir zorluk olduğunu vurgular. Bölüm V-E'ta bu bulgunun bizim deneylerimizde de sürdüğünü gösteriyoruz.

III. YÖNTEM

Bu bölümde FastGRNN hücreni, hücreye uygulanan üç aşamalı sıkıştırma boru hattını ve modelin çarpıcısız bir MCU üzerinde çalışabilmesini sağlayan LUT tabanlı aktivasyon yöntemini tanımlıyoruz. Şekil 1, tüm akışı özetler.

A. FastGRNN Hücre Formülasyonu

t anındaki giriş $\mathbf{x}_t \in \mathbb{R}^d$ ve önceki gizli durum $\mathbf{h}_{t-1} \in \mathbb{R}^H$ verildiğinde, FastGRNN hücresi [5] tek bir kapı $\mathbf{z}_t$, aday güncelleme $\tilde{\mathbf{h}}_t$ ve iki skalerli ara değerlendirme $\mathbf{h}_t$ hesaplar:

$$\mathbf{z}_t = \sigma(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}_z) \quad (1)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}_h) \quad (2)$$

$$\mathbf{h}_t = (\zeta(1 - \mathbf{z}_t) + \nu) \tilde{\mathbf{h}}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1} \quad (3)$$

Burada $\odot$ eleman bazında çarpmayı, $\zeta, \nu \in (0, 1)$ ise sızintılı-integratör davranışı ile standart kapılı güncelleme arasında denge kuran öğrenilmiş skalerleri gösterir. FastGRNN'yu ayıran nokta bu iki skalerli kapıdır: GRU ya da LSTM'deki üç veya dört ağırlık çifti yerine, kapı ve aday güncelleme aynı ağırlık çiftini $(\mathbf{W}, \mathbf{U})$ paylaşır. Böylece LSTM'ye yakın ifade gücü çok daha az parametreyle elde edilir. HAR kurulumumuzda $d=3$ (üç eksenli ivme), $H=16$ ve giriş dizisi uzunluğu $T=128$ örnektir (50 Hz'de 2.56 s'lik bir pencere). $H=16$

seçimi Tablo I'da deneysel olarak gerekçelendirilmiştir: gizli boyutu $H=32$ 'ye çıkarmak parametre sayısını yaklaşık üç kat artırdığı hâlde daha düşük test F1 üretir ve eğitim uzadıkça sonuç daha da kötüleşir. Bu davranış, daha küçük hücrenin veri kümesindeki yapıyı zaten yakaladığı ve büyük modelin aşırı uyuma gittiği anlamına gelir.

Tablo I

HAPT ÜZERİNDE GİZLİ BOYUT SEÇİMİ; TAM RANKLI FASTGRNN (DÜŞÜK RANK YOK, SEYREKLEŞTİRME YOK). $H=32$, $H=16$ 'YA GÖRE YAKLAŞIK ÜÇ KAT PARAMETREYE SAHİP OLMASINA RAĞMEN EŞLEŞEN HER YAPILANDIRMADA DAHA KÖTÜDÜR VE EPOCH 30'DAN 50'YE *daha* da BOZULUR; BU DURUM AŞIRI UYUMA İŞARET EDER. $H=16$ EPOCH 120'YE KADAR İYİLEŞMEYE DEVAM EDER (DOĞRULAMA-SEÇİMLİ TAVAN F1 = 0.912; BKZ. BÖLÜM V-A). $H=32$ 120 EPOCH'A UZATILMAMIŞTIR; 30 $\rightarrow$ 50 ARALIĞINDAKİ MONOTON BOZULMA, DAHA UZUN EĞİTİMİN FARKI KAPATMA OLASILIĞINI SON DERECE DÜŞÜRÜR. BU NEDENLE SONRAKİ TÜM DENEYLERDE $H=16$ KULLANILMIŞTIR.

H	Epoch	F1	Doğruluk	Parametre
16	30	0.847	0.854	440
16	50	0.861	0.869	440
16	120	0.912	0.913	440
32	30	0.833	0.845	1,384
32	50	0.830	0.834	1,384
32	120	—	—	1,384

Aynı ağırlık çifti $(\mathbf{W}, \mathbf{U})$ hem (1) hem de (2) içinde yeniden kullanıldığından, hücrenin kısıtsız parametre sayısı şöyledir:

$$\begin{aligned} \#params &= \underbrace{Hd}_{\mathbf{W}} + \underbrace{H^2}_{\mathbf{U}} + \underbrace{2H}_{\mathbf{b}_z, \mathbf{b}_h} + \underbrace{2}_{\zeta, \nu} \\ &= 48 + 256 + 32 + 2 = 338. \end{aligned} \quad (4)$$

Aynı gizli boyuttaki standart bir LSTM, bu tür dört ağırlık matrisi gerektirir ($\approx 1,280$ parametre). Dolayısıyla FastGRNN, mimari düzeyde $\sim 4\times$ üstünlükle başlar. Aşağıdaki düşük rank ve seyrekleştirme aşamaları bunu bir büyüklük mertebesi daha azaltır.

B. Düşük Ranklı Ağırlık Ayrıştırması

Giriş matrisi $\mathbf{W} \in \mathbb{R}^{H \times d}$ ve yinelemeli matris $\mathbf{U} \in \mathbb{R}^{H \times H}$, iki ince matrisin çarpımı olarak ayrıştırılır:

$$\mathbf{W} = \mathbf{W}_1 \mathbf{W}_2^\top, \quad \mathbf{W}_1 \in \mathbb{R}^{H \times r_w}, \mathbf{W}_2 \in \mathbb{R}^{d \times r_w}, \quad (5)$$

$$\mathbf{U} = \mathbf{U}_1 \mathbf{U}_2^\top, \quad \mathbf{U}_1 \in \mathbb{R}^{H \times r_u}, \mathbf{U}_2 \in \mathbb{R}^{H \times r_u}. \quad (6)$$

$\mathbf{W}_1, \mathbf{W}_2, \mathbf{U}_1, \mathbf{U}_2$ çarpanları birlikte öğrenilir. Yinelemeli matris-vektör çarpımı $\mathbf{U}\mathbf{h} = \mathbf{U}_1(\mathbf{U}_2^\top \mathbf{h})$ olarak değerlendirilir; böylece maliyet H^2 yerine $2Hr_u$ çarp-topla işlemine iner. $r_u \in \{4, 6, 8, 12\}$ değerlerini her biri beş rastgele tohumla taradıktan sonra (Bölüm V-B), $r_w = 2, r_u = 8$ seçilmiştir.

C. İteratif Sert Eşikleme

Dört çarpan matrisinin tamamını iteratif sert eşikleme [12] ile seyrekleştiriyoruz. Her eğitim adımında, her ağırlık tensöründeki mutlak değerce en büyük k girdi korunur; diğerleri sıfırlanır. Hedef seyreklik s , eğitim epoch'u e boyunca

$$s_e = s \cdot \min\left(1, \frac{e}{e_{\text{ramp}}}\right)^3 \quad (7)$$

küçük çizelgesini izler; burada $e_{\text{ramp}} = 50$ olup bunu 50 epoch'luk maske-sabit ince ayar izler. $s \in \{0.3, 0.5, 0.7, 0.9\}$ değerleri beş tohumla tarandıktan sonra (Bölüm V-C), doğruluk ile sıkıştırılabilirliği dengeleyen U-eğrisi optimumu olarak $s = 0.5$ (283 sıfır olmayan parametre) seçilmiştir.

D. Aktivasyon Kalibrasyonlu Tensör-Başına Q15 Nicelen-dirme

Her ağırlık tensörü $\mathbf{W}^{(\ell)}$ ($\ell \in \{W_1, W_2, U_1, U_2\}$) aşağıdaki biçimde nicelendirilir:

$$\hat{w}_{ij}^{(\ell)} = \text{clip}\left(\text{round}(w_{ij}^{(\ell)} / s_\ell), -2^{15}, 2^{15} - 1\right), \quad (8)$$

Burada tensöre özgü ölçek s_ℓ , $\max_{ij} |w_{ij}| / s_\ell$ değerinin Q15 tavanının hemen altında kalacağı biçimde seçilir. Çıkarım sırasında ileri geçişte çözülmüş değer $\hat{w}_{ij}^{(\ell)} \cdot s_\ell$ kullanılır.

Bu şemayı aktivasyonlara doğrudan Q15 $[-1, 1)$ biçiminde uygulamak yıkıcı sonuç verir: eğitilmiş modelimizde gizli durum $\mathbf{h}_t \sim 62$ büyüklüklerine ulaşır; bu değer Q15'in temsil edilebilir aralığının bir büyüklük mertebesi ötesindedir. Müdahale edilmediğinde F1 0.918'den 0.16'ya düşer ve STANDING sınıfı tamamen kaybolur (Bölüm V-D).

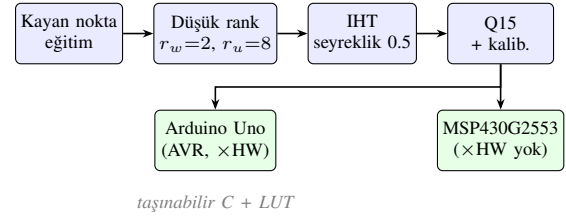
Basit bir çözüm, gözlenen $\mathbf{h}_t$ büyüklüklerini kapsayan ± 64 temsil aralığına sahip sabit Q9.6 sabit nokta biçimine geçmektir. Ancak bu seçim, her tensörün gerçek dinamik aralığını göz ardı ederek tüm tensörlerde aynı güvenlik payını kullanır ve çözünürlükten ödün verir. Bunun yerine *aktivasyon başına kalibrasyon* kullanıyoruz: kalibrasyon geçişi, eğitim verisinin beş mini-yığını FP32 modelden geçirir, her ara tensörün ampirik maksimumunu kaydeder, %10 güvenlik payı uygular ve her aktivasyona kendi ölçeğini atar. Tensör-başına şema Q9.6'yı genelleştirir; buna ihtiyaç duyan tensörler için (ör. $\mathbf{h}_t$) ± 64 aralığına uyarlanabilir biçimde yaklaşırken, buna ihtiyaç duymayan tensörlerde (ör. kapı çıktısı $\mathbf{z}_t \in [0, 1]$) tam Q15 çözünürlüğünü korur. Kalibrasyonla birlikte tamsayı C uygulaması ile FP32 referansı arasındaki tahmin uyumu 3,399 pencere test kümesinin tamamında %100'dür ve C tarafındaki makro F1 0.9176'ya ulaşır; bu değer makale boyunca raporlanan gömülü model sonucudur.

E. LUT Tabanlı Aktivasyonlar

Çarpıcısız bir hedefte en pahalı işlemler σ ve tanh hesaplarıdır. Her çağrı yazılımla öykünülen aşkın fonksiyonlara gider; hücre de örnek başına $2H$ aktivasyon hesaplar ($H=16$ için $2 \cdot 16 = 32$, 128 örneklili bir pencerede 4,096 çağrı). Çalışma zamanındaki bu çağrıları, Flash'ta saklanan ve $[-8, +8]$ giriş aralığında tanımlanan 256 girişli arama tablolarıyla değiştiriyoruz:

$$\text{LUT}[k] = f\left(-8 + k \cdot \frac{16}{255}\right), \quad k \in \{0, 1, \dots, 255\}, \quad (9)$$

Burada $f \in \{\sigma, \tanh\}$. $[-8, +8]$ dışındaki girişler ± 1 değerlerine doyurulur; bu bölgelerde her iki fonksiyon da kayan nokta duyarlığında zaten bu değerlere çok yakındır. Aralık içinde ise bitişik girdiler arasında doğrusal ara değerlendirme yapılır. Böylece her aktivasyon bir karşılaştırma, iki indeksli yükleme, bir çıkarma ve bir çarp-topla işlemine iner. İki tablo birlikte



Şekil 1. Uçtan uca FastGRNN sıkıştırma ve gömülü hedefe aktarma boru hattı. Kayan nokta eğitim → düşük rank → seyrek → kalibre edilmiş Q15 ağırlıklar → LUT aktivasyonlu taşınabilir C → Arduino ve MSP430 ikilileri.

1 KB Flash kaplar; bu değişiklik MSP430G2553 üzerinde uçtan uca **30.5**× hızlanma sağlar (Bölüm V-G).

F. Uygulama

Eğitim PyTorch 2.x ile yapılmıştır. Gömülü C çıkarım motoru (`fastgrnn.cpp`), hem `avr-gcc` (Arduino Uno R3) hem de Code Composer Studio `msp430-elf-gcc` (MSP430G2553) altında değiştirilmeden derlenen tek bir ~200 satırlık çeviri birimidir. Tüm ağırlıklar, tensör-başına ölçekler ve iki aktivasyon LUT'u ile birlikte Flash içinde `const int16_t` dizisi olarak saklanır. Çalışma zamanı çalışma kümesi (gizli durum $\mathbf{h}$, kapı $\mathbf{z}$, aday $\mathbf{h}$, çıkış logitleri ve geçici alan) toplamda ~300 bayttır ve MSP430G2553'nun 512 B SRAM bütçesine rahatça sığar.

IV. DENEYSEL KURULUM

A. Veri Kümesi

UCI HAR [10] veri kümesinin bir uzantısı olan Human Activities and Postural Transitions (HAPT) veri kümesini [11] kullanıyoruz. HAPT, belde taşınan bir akıllı telefonda 50 Hz'de kaydedilmiş üç eksenli doğrusal ivme verilerini içerir; 30 katılımcı altı temel aktiviteyi gerçekleştirir: WALKING, UPSTAIRS, DOWNSTAIRS, SITTING, STANDING ve LAYING. [11] çalışmasındaki kanonik özne-ayrık eğitim/doğrulama/test bölmesini benimsiyoruz. Bu bölme, uzunluğu 128 olan (50 Hz'de 2.56 s) 7,352 eğitim penceresi, 1,515 doğrulama penceresi ve 3,399 test penceresi üretir. Bu makalede raporlanan tüm doğruluk ve F1 değerleri ayrılmış test bölmesi üzerinde hesaplanmıştır.

Akış değerlendirmesinde her test penceresini 50 Hz'de örnek örnek besliyor ve her pencere sınırında gizli durumu sıfırlıyoruz. Böylece değerlendirme, gömülü çalışma zamanındaki davranışla birebir eşleşir.

B. Eğitim Protokolü

Modeller, masaüstü CPU üzerinde PyTorch 2.x ve Adam iyileştiricisi ($\eta=10^{-3}$, yığın boyutu 64, 100 epoch) ile eğitilmiştir. IHT seyrekleştirme maskesi Bölüm III-C'deki küçük çizelgeyi izler; hedef seyreklik epoch 50'de ulaşılır ve sonraki 50 epoch'luk ince ayar boyunca maske sabit kalır. Raporlanan her yapılandırma beş tohum $\{0, 1, 2, 3, 4\}$ üzerinde tekrarlanır; aksi belirtilmedikçe istatistikler ortalama $\pm$ standart sapma olarak verilmiştir.

C. Gömülü Hedef Donanım

İki bare-metal hedef değerlendirilmiştir:

- **Arduino Uno R3** (ATmega328P): 16 MHz’de çalışan 8-bit AVR çekirdeği; 32 KB Flash; 2 KB SRAM; donanım 8×8 çarpıcısı; kayan nokta birimi yok. Araç zinciri: `-Os` seçeneğiyle `avr-gcc` kullanan Arduino IDE 2.3.x.
- **MSP430G2553**: kalibre edilmiş 1 MHz DCO’da çalışan 16-bit MSP430 çekirdeği; 16 KB Flash; 512 B SRAM; hiçbir tür donanım çarpıcısı yok; FPU yok. Araç zinciri: `-O2` ile derlenen `TI msp430-elf-gcc` bare-metal derlemesini kullanan Code Composer Studio 12.x. Energia çalışma zamanı kullanılmamıştır.

Her iki hedefte de yüklenen program şu bileşenlerden oluşur: (i) `fastgrnn.cpp` çıkarım motoru, (ii) Q15 ağırlık tablosu ve tensör-başına ölçekler (`model_weights.h`), (iii) 256 girişli sigmoid ve tanh arama tabloları ve (iv) 9600 (MSP430G2553) ya da 115,200 baud (Arduino Uno R3) hızında sınıf etiketi gönderen UART sürücüsü. Geri kalan Flash alanı kullanılmaz.

D. Platformlar Arası Doğrulama Protokolü

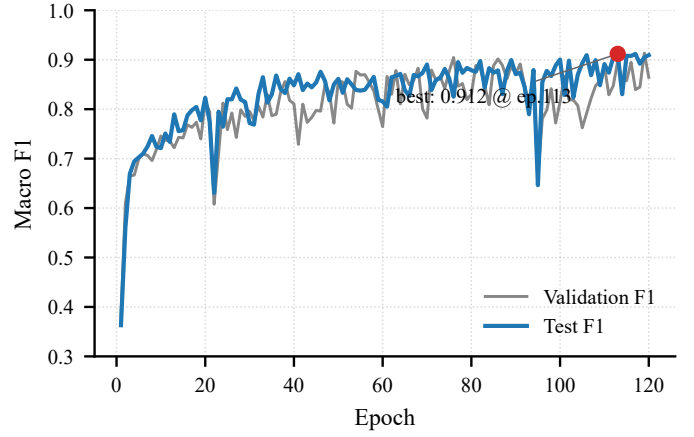
FP32 PyTorch referansı ile Q15 C uygulaması arasında %100 tahmin uyumu iddiasını doğrulamak için (Bölüm V-F), aynı girdileri iki çıkarım yolundan geçirip sonuçları karşılaştırıyoruz. Bir Python denetim betiği (`test_inference_python.py`) gömülü hedefe aktarılabilecek Q15 ağırlıkları yükler, NumPy içinde C ile eşdeğer bir ileri geçiş çalıştırır (LUT aktivasyonları ve tensör-başına ölçek çözme adımları dahil) ve her test penceresi için `argmax` tahminlerini FP32 referansı ile karşılaştırır. Arduino ve MSP430 üzerindeki C uygulaması da pencere-başına tahminleri UART üzerinden gönderir; bu tahminler aynı referansla denetlenir. Üç yürütme yolu – FP32 PyTorch, NumPy’deki C-eşdeğer uygulama ve bare-metal C – 3,399 pencerenin tamamında aynı sınıfı üretir.

E. Canlı Sensör Yapılandırması

Canlı gösterimler için InvenSense MPU-6050 sunan bir GY-521 modülü I²C üzerinden bağlanır. MSP430G2553 üzerinde USCI_B0 doğrudan sürülür (P1.6 = SCL, P1.7 = SDA, Energia Wire katmanı yoktur); Arduino Uno R3 üzerinde standart Wire kütüphanesi kullanılır. FP32 eğitim verisi ölçeklemesiyle eşleşmesi için yalnızca ± 2 g aralığındaki ivmeölçer okunur. Yazılım zamanlayıcısı örnekleme döngüsünü 50 Hz’e ayarlar; çıkarım adımından sonra örnek başına 7–11 ms zaman payı kalır (Bölüm V-G).

V. SONUÇLAR

Sonuçları L-S-Q boru hattındaki sıraya göre raporluyoruz (Bölüm V-A–V-D); ardından gömülü hedeflerdeki çalışma davranışına geçiyoruz (Bölüm V-F–V-G). Tüm deneyler Bölüm IV’deki protokolü kullanır.



Şekil 2. $H=16$ gizli boyutu için eğitim yörüngesi. Test F1 epoch 100 civarında doyar; en iyi tohum kontrol noktası (epoch 113’te $F1 = 0.912$), sonraki tüm sıkıştırma aşamaları için kullanılan temel modeldir.

Tablo II
HAPT ÜZERİNDE ARDIŞIK L-S-Q BORU HATTI. HER SATIR, BELİRTİLEN DÖNÜŞÜMÜ BİRİKİMLİ OLARAK UYGULAR. ORTALAMA F1 BEŞ TOHUM ÜZERİNDİR; SON Q15 SATIRI GÖMÜLÜ HEDEFE AKTARILAN SEED-0 MODELİ ÜZERİNDİR.

Aşama	F1	Sıfır olmayan	Boyut (FP32 / Q15)
FastGRNN tam rank ($H=16$)	0.912	440	1.8 KB
+ Düşük rank ($r_w=2, r_u=8$)	0.879	430	1.7 KB
+ Seyreklik ($s=0.5$)	0.856	283	1.1 KB
+ Q15 (ağırlık + kalib. akt.)	0.918	283	566 B

A. L-S-Q Boru Hattı Doğruluğu

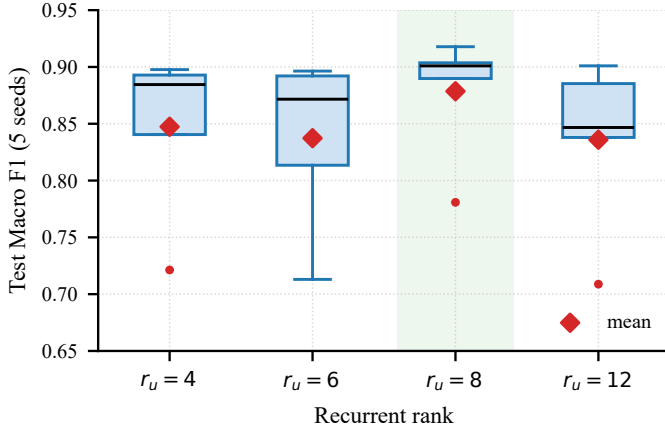
Tablo II, her sıkıştırma aşamasının birikimli etkisini özetler. Sınıflandırıcı başlığıyla birlikte tam ranklı FastGRNN hücresi (Bölüm III-A) toplam 440 parametreye sahiptir ve test kümesinde $F1 = 0.912$ değerine ulaşır. Düşük ranklı ayrıştırma ($r_w=2, r_u=8$), yinelemeli matris parametre sayısını $U = U_1 U_2^T$ ayrıştırması yoluyla azaltır (beş tohum üzerinde ortalama F1, 430 sıfır olmayan parametrede 0.879’a düşer). $s=0.5$ düzeyindeki seyrekleştirme, başka bir sınırlı düşüşle (0.856 ortalama) sayıyı 283 sıfır olmayan parametreye indirir. Bölüm III-D’deki aktivasyon kalibrasyonu ile tensör-başına Q15 nicelendirme ise doğruluğu *tam olarak geri kazanır*: 3,399 test penceresinde tamsayı C uygulaması PyTorch referansı ile tahminlerin %100’ünde uyuşur ve C tarafındaki makro F1 **0.9176** olur. Şekil 2, sonraki tüm aşamalara temel oluşturan $H=16$ eğitim çalışmasını gösterir.

Parametre bütçesini karşılaştırmalı görmek için Tablo III, gömülü hedefe aktarılan modeli aynı HAPT bölgesinde eğitilmiş bir MLP taban çizgisiyle (en hafif yinelemeli olmayan referans) ve aynı gizli boyuttaki LSTM ile GRU hücrelerinin kuramsal parametre sayılarıyla karşılaştırır. Gömülü FastGRNN LSQ modeli, MLP taban çizgisinden **44× daha küçük** ve eşleşen H değerinde, sınıflandırıcı ek yükünden önce, bir LSTM hücresinden **~5× daha küçüktür**.

Tablo III

GÖMÜLÜ HEDEFE AKTARILAN MODELİN, YİNELEMELİ OLMAYAN VE YİNELEMELİ TABAN ÇİZGİLERİNE GÖRE PARAMETRE AYAK IZI. LSTM VE GRU SAYILARI $H=16$, $d=3$ İÇİN KURAMSALDIR (YALNIZCA HÜCRE); İLİŞKİLİ HAR GÖREVLERİNDE RAPORLANAN DOĞRULUKLARI FASTGRNN İLE BENZER YA DA BIRAZ DAHA YÜKSEKTİR, ANCAK $\sim 4\times$ PARAMETRE DEZAVANTAJI TAŞIRLAR [5]. MLP F1 BU ÇALIŞMADA ÖLÇÜLMÜŞTÜR; SINIFLANDIRICI BAŞLIĞI HER DURUMDA 102 PARAMETRE EKLER.

Model	Param. (hücre)	HAPT F1
MLP taban çizgisi	12,518 (tam)	0.847
LSTM ($H=16$, kuramsal)	1,280	—
GRU ($H=16$, kuramsal)	960	—
FastGRNN hücresi ($H=16$)	338	—
FastGRNN L (düşük rank)	328	0.879
FastGRNN LSQ (gömülü)	181	0.918



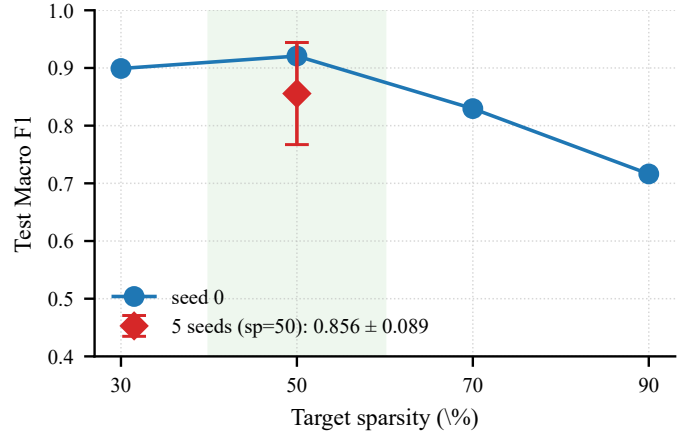
Şekil 3. Yinelemeli rank seçimleri boyunca tohum-başına test F1. $r_u=8$ en iyi ortalamaya ve kabul edilebilir varyansa ulaşır. Kırmızı elmaslar rank-başına ortalamaları; yeşil bant seçilen yapılandırmayı gösterir.

B. Çok Tohumlu Kararlılık ve Rank Seçimi

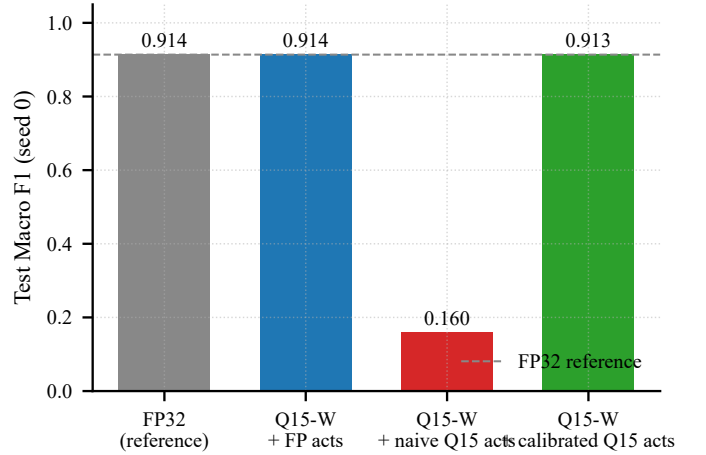
Şekil 3, yinelemeli rank değerleri $r_u \in \{4, 6, 8, 12\}$ için her biri beş tohumdan oluşan test-F1 dağılımını raporlar. $r_u=8$ en iyi ortalamaya (0.879 ± 0.056) ve en dar kuyruğa ulaşır. Dört yapılandırmanın tamamında seed 1, ranktan bağımsız olarak tutarlı biçimde düşük bir aykırı değerdir (~ 0.71 F1). Bu, özellikle vurgulanması gereken bir yeniden üretilebilirlik bulgusudur: tek bir şanssız başlatma, küçük örneklemli bir değerlendirmeyi yanıltabilir. Bu model boyutunda en az beş tohum üzerinde ortalama $\pm$ standart sapma raporlamak bu nedenle isteğe bağlı değildir.

C. Seyreklik Taraması

Şekil 4, sabit $r_u=8$ altında hedef seyreklik $s \in \{0.3, 0.5, 0.7, 0.9\}$ değerlerini tarar. Tek tohumlu tarama sığ ancak görünür bir U-eğrisi çizer: $s=0.5$ en fazla doğruluğu korur, $s=0.3$ eğitim maliyetini haklı çıkaracak kadar sıkıştırılmaz ve $s \geq 0.7$ belirgin bozulmaya yol açar. Gömülü uygulama için en uygun nokta olan $s=0.5$ değerinde beş tohumlu ortalama 0.856 ± 0.099 'dur; en yüksek tohum 0.92'ye ulaşır. $s=0.5$ değerindeki yüksek standart sapma, seyrekleştirilmemiş



Şekil 4. Hedef seyreklik karşısında test F1. Tek tohumlu tarama (mavi çizgi) ve $s=0.5$ için beş tohumlu ortalama $\pm$ standart sapma (kırmızı elmas). 0.4–0.6 bandı gömülü uygulama için en uygun aralıktır.



Şekil 5. Seed-0 modelinde nicelendirme kipleri. Yalnızca ağırlık Q15 kayıpsızdır; saf Q15 aktivasyonları modeli çökertir; kalibre edilmiş Q15 aktivasyonları referansı yuvarlama gürültüsü içinde geri kazanır.

ortalamaya (0.879 ± 0.056) göre bilgi-kuramsal sınırına yakın bir parametreleştirme ile çalışmanın bedelidir.

D. Nicelendirme Kipleri

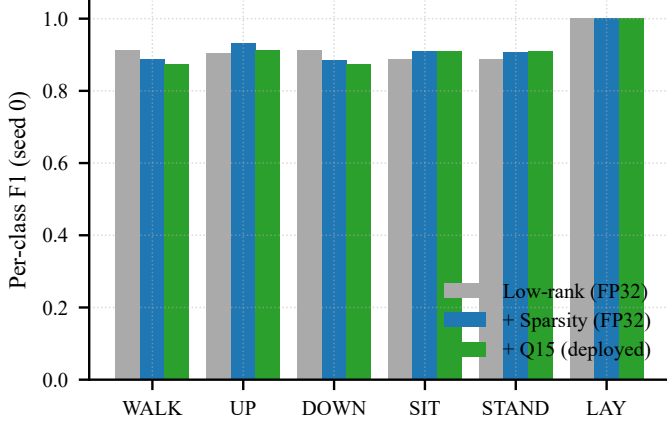
Şekil 5 ve Tablo IV, seed-0 modelinde dört nicelendirme yapılandırmasını karşılaştırır. Yalnızca ağırlık Q15 kayıpsızdır: σ ve tanh FP32'de çalışmayı sürdürür ve ağırlık nicelendirme gürültüsü, eğitim yordamının kendi tohumlar arası varyansının altında kalır. Kalibrasyonsuz saf Q15 aktivasyon nicelendirmesi yıkıcıdır: F1 0.918'den 0.16'ya çöker ve STANDING sınıfı tamamen kaybolur. Beş eğitim mini-yığınının oluşan deterministik bir ön geçiş olan tensör-başına aktivasyon kalibrasyonu Q15 çıkarımı, FP32 referans doğruluğunu her sınıfta yuvarlama gürültüsü düzeyinde geri kazanır. Her iki MCU'ya aktarılan yapılandırma budur.

E. Sınıf-Başına Davranış

Şekil 6, üç FP32 boru hattı aşaması ve gömülü Q15 model boyunca sınıf-başına F1 değerlerini izler. Statik sınıflar

Tablo IV
NİCELENDİRME KIPININ SEED-0 TEST F1 ÜZERİNDEKİ ETKİSİ.

Kip	F1	Notlar
Float32	0.918	Referans
Q15 ağırlık, FP32 akt.	0.918	Kayıpsız
Q15 ağırlık, <i>saf</i> Q15 akt.	0.16	STANDING çöküşü
Q15 ağırlık, <i>kalibre</i> Q15 akt.	0.918	Konuşlandırılan



Şekil 6. Sıkıştırma aşamaları boyunca sınıf-başına F1 (seed 0). LAYING kolayca ayrılabilir; DOWNSTAIRS ise geniş literatürü yansıtarak süreç boyunca en zor sınıf olarak kalır.

(SITTING, STANDING, LAYING) yüksek F1 değerlerini korur. WALKING ve UPSTAIRS, seyrekleştirme altında 1–3 yüzde puanı bozulur; ancak 0.88’in üzerinde kalır. DOWNSTAIRS, tüm süreç boyunca performansı belirleyen en zor sınıftır: düşük rank FP32’de ~ 0.91 ’den seyrek modelde 0.89’a düşer ve kalibre edilmiş Q15 sonrasında 0.91’e geri döner. Bu bulgu, akıllı telefon düzeyi ivmeölçerlerde dinamik iniş hareketini sistematik olarak en zor sınıf olarak tanımlayan geniş HAR literatürüyle tutarlıdır [25], [26].

F. Platformlar Arası Deterministik Çıkarım

Aynı taşınabilir C kaynak dosyası, hem 8-bit Arduino Uno R3 hem de 16-bit MSP430G2553 üzerinde değiştirilmeden derlenir. 3,399 pencere test kümesinin tamamında tahminler olayların %100’ünde uyuşur; logitler ise mutlak olarak 10^{-2} değerinden daha iyi düzeyde uyuşur (herhangi bir karar sınırı için anlamlı farkın oldukça altında). Tablo V, tek bir akış test penceresinde altı zaman noktasında gizli bileşen h_0 değerini örnekler: iki platform iki ondalık basamağa kadar aynı yörüngeleri üretir. Bu açık bir sonuç değildir. İki aygıt farklı yazılım öykümleri kayan nokta uygulamaları, farklı çağrı kuralları ve farklı sözcük boyutları kullanır. Buna rağmen uçtan uca tahminin bit-eşdeğer olması, kalibre edilmiş Q15 ağırlıklar ile FP32’de biriktirip sonra doyuran aritmetikimizin uygulama ayrıntılarına karşı kararlı olduğunu gösterir. Bu, güvenlikle ilişkili HAR için korunmaya değer bir yeniden üretilebilirlik özelliğidir.

Tablo V
50 HZ ZAMANLAMALI TEK BİR AKIŞ TEST PENCERESİNDE PLATFORMLAR BOYUNCA GİZLİ DURUM BİLEŞENİ h_0 EVRİMİ.

t (örnek)	Arduino h_0	MSP430 h_0
25	-0.720	-0.720
50	-0.352	-0.352
75	+0.459	+0.459
100	+3.803	+3.803
125	+11.388	+11.388
128	+12.542	+12.542

Tablo VI
50 HZ ZAMANLAMALI AKIŞ PERFORMANSI. BÜTÇE 20 MS/ÖRNEKTİR. HER İKİ PLATFORMDA DA 256 GİRİŞLİ LUT ETKİNDİR.

Platform	Ort.	Maks.	Bütçe	Aşım
Arduino Uno R3	9.21	10	46%	0 / 128
MSP430G2553	13.0	14	65%	0 / 128

G. Gerçek Zamanlı Akış Performansı

Tablo VI, 50 Hz zamanlamalı akış altında (20 ms bütçe) örnek-başına gecikmeyi raporlar. Her iki platform da 128 örnekli test penceresinin tamamında bütçeyi aşan örnek olmadan gerçek zamanlı çalışır. Arduino ortalamada bütçenin %46’sını kullanır ve I²C okumaları, LED geri bildirimi ve UART günlüklemesi için geniş alan bırakır. MSP430G2553 ortalamada %65 kullanır ve yine de bütçenin rahatça altında kalır.

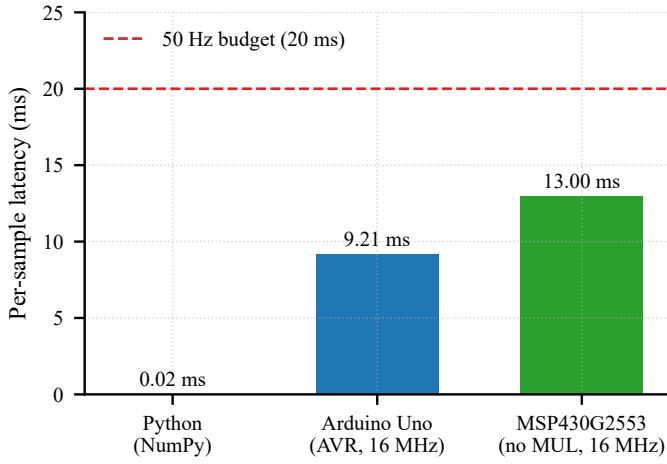
Bölüm III-E’deki 256 girişli sigmoid/tanh LUT, MSP430G2553’ü uygulanabilir kılan unsurdur. LUT olmadan MSP430G2553 üzerinde tam pencere çıkarımı ~ 54 s sürer; LUT ile bu süre ~ 1.8 s’ye iner ve $30.5\times$ hızlanma sağlar. Akış rejimine çevrildiğinde bu, tamamen gerçek zamanlı olmayan bir çalışma ile örnek başına yedi milisaniyelik zaman payı arasındaki farktır. Aynı LUT Arduino Uno R3 üzerinde de etkindir; AVR çekirdeğinde zaten donanım çarpıcısı bulunduğu için buradaki hızlanma daha sınırlıdır ($1.51\times$).

VI. TARTIŞMA

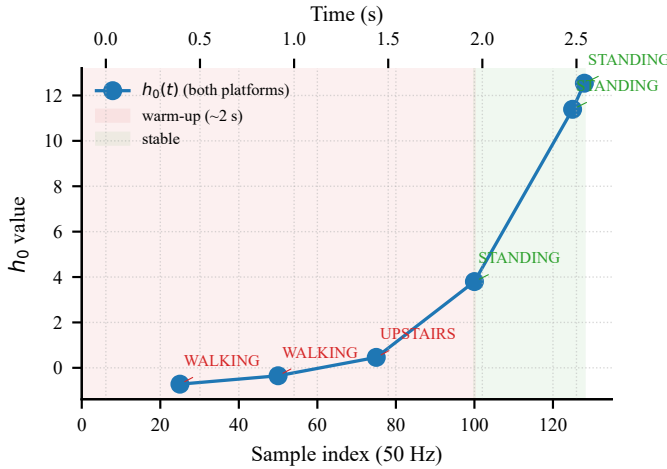
A. Yinelemeli Isınma Gecikmesi

Özgün FastGRNN makalesinde [5] ele alınmayan önemli bir nokta şudur: sınıf etiketinin kararlı hâle gelmesi için, kullanılan donanımdan bağımsız olarak, yaklaşık iki saniyelik (~ 100 örnek, 50 Hz) gizli durum geçmişi gerekir. Şekil 8, tek bir STANDING penceresi boyunca üretilen etiketi ve h_0 bileşenini izler. Model, $t=100$ civarında doğru sınıfa kilitlenmeden önce WALKING ve UPSTAIRS tahminlerinden geçer. Bu davranış iki MCU’dan birine özgü değildir; Arduino ve MSP430 aynı yörüngeleri üretir (Bölüm V-F).

Düşme algılama, jest tanıma ya da duruş koçluğu gibi kullanıcıya dönük uygulamalarda bu bulgu önemlidir. Aktivite geçişleri, ancak gerçekleştikten yaklaşık 2 s sonra güvenle doğrulanabilir. Bu kayda değer bir gecikmedir; cihaz-üstü RNN’ler değerlendirilirken örnek-başına hızın yanında ayrıca raporlanmalıdır. Benzer büyüklükteki başka kapılı yinelemeli



Şekil 7. Üç referans platformda örnek-başına çıkarım gecikmesi. Kırmızı kesikli çizgi, 50 Hz örneklemenin getirdiği 20 ms bütçeyi gösterir. Her iki MCU hedefi de bütçenin belirgin biçimde altında çalışır.



Şekil 8. Tek bir 2.56 s pencerede (50 Hz, STANDING gerçek etiketi) gizli durum bileşeni h_0 ve üretilen sınıf etiketi. Arduino ve MSP430 aynı yörüngeleri üretir. Yaklaşık 2 s geçmiş birikene kadar sınıf etiketi kararsızdır.

hücrelerde de aynı etkinin görülebileceğini düşünüyoruz. Bunu eşleşen parametre sayılarında LSTM/GRU taban çizgileriyle sınamak doğal bir devam çalışmasıdır.

B. Yeniden Üretilabilirlik Olarak Platformlar Arası Determinizm

Modern ML çıkarımı, donanımlar arasında çoğu zaman birebir yeniden üretilebilir değildir. Üreticiye özgü BLAS uygulamaları, karma duyarlılık tensör çekirdekleri ve birleşmeli olmayan kayan nokta indirgemeleri küçük farklar doğurabilir. Buna karşılık taşınabilir C motorumuz, hem 8-bit AVR hem de 16-bit MSP430 hedeflerinde yazılımla öykünülen kayan nokta kullanmasına rağmen platformlar arasında *bit-eşdeğer* tahminler üretir (Bölüm V-F). Bu düzeyde determinizm, tıbbi nitelikli HAR gibi güvenlikle ilişkili gömülü uygulamalarda korunması gereken bir özelliktir. Düzenleyici kurumlar, bir yazılım değişikliğinin yalnızca yetkilendirildiği donanım üzerinde yeniden doğrulanmasını isteyebilir. İki farklı ISA üze-

rinde bit-eşdeğer kalan bir çıkarım yolu, bu kısıtı hafifletmek açısından değerlidir.

C. Zaman Payı Analizi ve Saf-Q15 Çıkarması

MSP430G2553, örnek başına 20 ms bütçenin %65'ini kullanır ve ~7 ms zaman payı bırakır (Bölüm V-G). Canlı MPU-6050 testlerinde bu pay, örnek başına bir I²C imveölçer okumasını, bir LED durum güncellemesini ve bir UART günlük satırını rahatça karşılar.

Daha agresif bir *saf-Q15* çıkarım yolu da denendi: ağırlıklar, aktivasyonlar ve ara çarpımların tamamı Q15 tamsayı aritmetiğinde tutuldu. Bu yol pratikte başarısız oldu. Çok aşamalı düşük ranklı çarpım sırasında tensör-başına ölçekler zincirlenerek $\sim 10^{-13}$ düzeyine kadar küçülüyor; bu değer int64 biriktirici ve özel bir aşama-başına kaydırma çizelgesi olmadan Q15 çarpıcıda temsil edilemiyor. Bölüm III-D'de tartışılan sabit Q9.6 biçimi ölçek zincirleme sorununu hafifletir; ancak bu kez ağırlık tarafında çözünürlük kaybı yaratır. Ağırlık büyüklükleri tensörler arasında dört büyüklük mertebesine yayıldığı için, tek bir Q9.6 ölçeği bazı tensörlerde kalibre edilmiş yola göre $8 \times$ 'a varan çözünürlük kaybına neden olur. Daha temiz çözüm, nicelendirme farkında eğitimidir [20]; çünkü ölçek çizelgesi eğitim sonrasında zorlanarak değil, eğitim sırasında öğrenilir. LUT tabanlı FP32 yol 50 Hz gerçek zamanlı kısıtı yeterli zaman payıyla karşıladığı için bu çalışmada QAT yolunu sürdürmedik. Bunu gelecek çalışma olarak ele alıyoruz (Bölüm VI-E).

D. Sınırlılıklar

İddialarımızın kapsamını dört sınırlılık belirler:

- **Veri kümesi.** HAPT, sabit biçimde bele takılmış bir akıllı telefonla laboratuvar koşullarında kaydedilmiştir. Sensörün vücut üzerindeki yer değiştirmesi, serbest yaşam verilerindeki dağılım kayması ve özneler arası değişkenliğin HAR doğruluğunu düşürdüğü bilinmektedir [25], [26]; bunlar bu çalışmada modellenmemiştir.
- **Etiket kümesi.** Model altı temel HAPT aktivitesi üzerinde eğitilmiş ve değerlendirilmiştir. Tam HAPT etiket kümesi altı postüral geçiş daha içerir (ör. STAND-TO-SIT, LIE-TO-SIT); modelimiz bu geçişleri işlemez. Bu tercih, özgün FastGRNN HAR kıyaslamasıyla eşleşmek için yapılmıştır.
- **DOWNSTAIRS sınıfı.** DOWNSTAIRS, boru hattı boyunca performansını belirleyen en zor sınıftır (Şekil 6). Bu makalede sınıfa özgü bir iyileştirme denemesi yapılmamıştır. Bölüm VI-E olası birkaç müdahaleyi özetler.
- **İstatistiksel anlamlılık testleri.** Çok tohumlu değerlendirmede beş tohum üzerinde ortalama $\pm$ standart sapma raporlanmıştır; ancak eşleştirilmiş anlamlılık testleri ya da parametrik olmayan bootstrap aralıkları verilmemiştir. Biçimsel anlamlılık testi gelecek çalışmaya bırakılmıştır.

E. Gelecek Yönelimler

Bu çalışmanın ardından izlenebilecek beş doğal yön vardır.

- **Statik ve dinamik örüntüler için çift-rank ayrıştırma.** Rank taramamızda (Bölüm V-B) $r_u=4$ dinamik sınıflar (WALKING, UPSTAIRS, DOWNSTAIRS) için, $r_u=8$

ise statik sınıflar (SITTING, STANDING) için daha elverişlidir. Düşük maliyetli bir varyant olarak $U_{\text{eff}} = \text{LowRank}(r=4) + \text{diag}(\alpha)$ kullanılabilir. Bu yapı yalnızca H ek parametre ekler; statik DC-benzeri sinyalin diyagonal artık üzerinden, dinamik örüntünün ise düşük rank kolu üzerinden geçmesine izin verir. Bu yaklaşımın, statik sınıflarda bozulma yaratmadan $r_u=4$ 'ün dinamik sınıf başarımını geri kazanmasını bekliyoruz.

- **DOWNSTAIRS hedefli öznitelikler.** Standart UCI HAR boru hattı, yerçekimini vücut ivmesinden ayırmak için 0.3 Hz'de Butterworth alçak geçiren süzgeç uygular. Özgün FastGRNN HAR kurulumuyla eşleşmek için bu adımı kullanmadık. Bu süzgeci yeniden eklemek ya da 128 örnekleli pencere üzerinde az sayıda FFT-bant özniteliğiyle girdiyi genişletmek, DOWNSTAIRS sınıfındaki farkı kapatmanın düşük maliyetli bir yolu olabilir.
- **Saf-Q15 çıkarım için nicelendirme farkında eğitim.** Bölüm VI-C'da tartışıldığı gibi, ölçek zincirleme nedeniyle eğitim-sonrası nicelendirme tam tamsayı boru hattında başarısız olur. QAT [20] bu sorunu çözebilir. Saf-Q15 yolun MSP430G2553 üzerinde ek $2-3\times$ hızlanma sağlaması muhtemeldir ve LUT tabanlı yaklaşımın doğal devamıdır.
- **Geçiş durumları.** Tam HAPT etiket kümesindeki altı postürel geçişi eklemek, yinelemeli durumun uzun ufuklu aktivite tanıma ile kısa ufuklu geçiş algılama arasında nasıl paylaştırılacağı sorusunu doğurur. İki farklı örnekleme temposunda değerlendirilen çok ölçekli bir yinelemeli hücre, makul bir başlangıç noktasıdır.
- **Çıkarım başına enerji.** Bu çalışma çıkarım-başına zaman raporlar; ancak çıkarım-başına joule raporlamaz. Kart üzerinde doğrudan akım ölçümü, gömülü uygulama değerlendirmesini tamamlar ve Bölüm I'daki motivasyona geri bağlanır. Silikon arzı ve enerji kısıtları birlikte düşünülmelidir; $30\times$ çalışma süresi hızlanması, uygulama düzeyinde ancak pil tüketimini de orantılı biçimde azaltıyorsa anlamlıdır.

VII. SONUÇ

Bu çalışmada FastGRNN algoritmasını iki bare-metal mik-rodenetleyici üzerinde uçtan uca yeniden ürettik: 8-bit Arduino Uno R3 ve donanım çarpıcısı bulunmayan 16-bit MSP430G2553. Sonuçlar, kompakt kapılı yinelemeli hücrelerin kilobayt düzeyinde belleğe sahip giyilebilir HAR uygulamaları için pratik bir seçenek olduğunu gösterir. Gömülü hedefe aktarılan modelin ağırlıkları 566 bayt yer kaplar; model HAPT kıyaslamasında makro $F1 = 0.918$ değerine ulaşır ve her iki hedefte de bütçeyi aşan örnek olmadan 50 Hz'de gerçek zamanlı çalışır. Tamsayı C çıkarım motoru, 8-bit AVR ve 16-bit MSP430 uygulamaları arasında bit-eşdeğer sonuç verir.

Çalışma, özgün FastGRNN makalesini üç yönden genişletir: platformlar arası deterministik çıkarım, çarpıcısız MCU'lar için $30.5\times$ LUT tabanlı hızlanma ve uçtan uca yanıt süresini belirleyen ~ 2 s yinelemeli ısınma gecikmesinin deneysel olarak ölçülmesi. Bu sonuçlar daha genel bir noktaya işaret eder: küresel yarı iletken arzının ML talebini sınırsız biçimde

daha büyük donanımlarla karşılayamayacağı bir dönemde, yinelemeli mimarileri algoritmik olarak küçültmek güçlü bir alternatiftir. Başka bir ifadeyle, yapay zekâyı zaten elimizde olan silikon üzerinde çalıştırmak mümkündür. Tüm kod, modeller ve yeniden üretim dosyaları Apache Lisansı 2.0 altında <https://github.com/emre1998/fastgrnn-har> adresinde paylaşılmıştır.

KAYNAKLAR

- [1] O. Burkack, S. Lingemann, and K. Pototzky, "Coping with the auto-semiconductor shortage: Strategies for success," McKinsey & Company, 2022, <https://www.mckinsey.com/industries/automotive-and-assembly/our-insights>.
- [2] S. M. Khan, A. Mann, and D. Peterson, "The semiconductor supply chain: Assessing national competitiveness," Center for Security and Emerging Technology (CSET), Georgetown University, Tech. Rep., 2021.
- [3] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
- [4] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.
- [5] A. Kusupati, M. Singh, K. Bhatia, A. Kumar, P. Jain, and M. Varma, "FastGRNN: A fast, accurate, stable and tiny kilobyte sized gated recurrent neural network," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [6] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, "MLPerf tiny benchmark," in *Conference on Machine Learning and Systems (MLSys)*, 2021.
- [7] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for ARM Cortex-M CPUs," in *arXiv preprint arXiv:1801.06601*, 2018.
- [8] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Conference on Machine Learning and Systems (MLSys)*, 2021.
- [9] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [10] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *European Symposium on Artificial Neural Networks (ESANN)*, 2013.
- [11] J.-L. Reyes-Ortiz, L. Oneto, A. Samà, X. Parra, and D. Anguita, "Transition-aware human activity recognition using smartphones," *Neurocomputing*, vol. 171, pp. 754–767, 2016.
- [12] T. Blumensath and M. E. Davies, "Iterative hard thresholding for compressed sensing," *Applied and Computational Harmonic Analysis*, vol. 27, no. 3, pp. 265–274, 2009.
- [13] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [14] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *International Conference on Learning Representations (ICLR)*, 2016.
- [15] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [16] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.
- [17] A. Kumar, S. Goyal, and M. Varma, "Resource-efficient machine learning in 2 KB RAM for the internet of things," in *International Conference on Machine Learning (ICML)*, 2017.
- [18] C. Gupta, A. S. Suggala, A. Goyal, H. V. Simhadri, B. Paranjape, A. Kumar, S. Goyal, R. Udupa, M. Varma, and P. Jain, "ProtoNN: Compressed and accurate kNN for resource-scarce devices," in *International Conference on Machine Learning (ICML)*, 2017.

- [19] Microsoft Research India, “EdgeML: Machine learning for resource-constrained edge devices,” 2017–2024, <https://github.com/microsoft/EdgeML>.
- [20] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, “Quantized neural networks: Training neural networks with low precision weights and activations,” *Journal of Machine Learning Research*, vol. 18, pp. 1–30, 2017.
- [21] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning both weights and connections for efficient neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [22] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [23] A. Capotondi, M. Rusci, M. Fariselli, and L. Benini, “CMix-NN: Mixed low-precision CNN library for memory-constrained edge devices,” *IEEE Transactions on Circuits and Systems II*, 2020.
- [24] F. J. Ordóñez and D. Roggen, “Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition,” *Sensors*, vol. 16, no. 1, 2016.
- [25] J. Wang, Y. Chen, S. Hao, X. Peng, and L. Hu, “Deep learning for sensor-based activity recognition: A survey,” *Pattern Recognition Letters*, vol. 119, pp. 3–11, 2019.
- [26] F. Demrozi, G. Pravadeh, A. Bihorac, and P. Rashidi, “Human activity recognition using inertial, physiological and environmental sensors: A comprehensive survey,” *IEEE Access*, vol. 8, pp. 210816–210836, 2020.
- [27] Texas Instruments, “MSP430x2xx family user’s guide (slau144j),” 2013, <https://www.ti.com/lit/ug/slau144j/slau144j.pdf>.

EK A

HİPERPARAMETRELER

Tablo VII, gömülü hedefe aktarılan modeli üretmek için kullanılan hiperparametrelerin tamamını listeler. Tüm değerler beş eğitim tohumu boyunca değişmeden tutulmuştur.

Tablo VII
GÖMÜLÜ HEDEFE AKTARILAN FASTGRNN MODELİ İÇİN EĞİTİM VE AKTARIM HİPERPARAMETRELERİ.

Parametre	Değer
Gizli boyut H	16
Giriş boyutluluğu d	3 (üç eksenli ivme)
Pencere uzunluğu T	128 örnek (2.56 s)
Örnekleme hızı	50 Hz
Çıkış sınıfları	6
Yinelemeli rank r_u	8
Giriş rankı r_w	2
Hedef sıklık s	0.5 (283 sıfır olmayan / 566)
Seyreklik rampa epoch’u	50 (kübik çizelge)
Sabit-maskeli ince ayar epoch’u	50
İyileştirici	Adam
Öğrenme oranı η	10^{-3}
Yığın boyutu	64
Toplam epoch	100
Rastgele tohumlar	{0, 1, 2, 3, 4}
Aktivasyon fonksiyonu	sigmoid, tanh
LUT boyutu	her biri 256 giriş
LUT giriş alanı	$[-8, +8]$
Ağırlık nicelendirme	tensor-başına Q15
Aktivasyon nicelendirme	tensor-başına Q15 + kalibrasyon
Kalibrasyon mini-yığımları	5
Kalibrasyon güvenlik payı	10%

EK B

Q15 NİCELENDİRME UYGULAMASI

Bölüm III-D’de kullanılan tensor-başına Q15 nicelendirme adımı, her tensor için tek satırlık bir işlemdir. Python’da:

```
def to_q15(W: np.ndarray, scale: float) -> np.ndarray:
    return (W / scale).round() \
        .clip(-32768, 32767) \
        .astype(np.int16)
```

Tensör-başına ölçek

$$s_\ell = \max_{ij} |W_{ij}^{(\ell)}| / 32767,$$

olarak seçilir; bu, her nicelendirilmiş girdiyi işaretli 16-bit aralık içinde tutan en büyük ölçektir. Aynı ifade hem yinelemeli ağırlık çarpmaları $\mathbf{W}_1, \mathbf{W}_2, \mathbf{U}_1, \mathbf{U}_2$ hem de sınıflandırıcı başlığı için kullanılır. Çalışma zamanında çözülmüş değer şu şekilde hesaplanır:

```
float w = (float)W_q15[i] * scale;
```

Her çözme işleminin çalışma zamanı maliyeti bir int-to-float dönüşümü ve bir çarpmadır. Bunlar AVR üzerinde ucuzdur; LUT aktivasyonları araya girdiğinde çarpıcısız MSP430G2553 üzerinde de yönetilebilir düzeydedir.

EK C

LUT ÜRETİM ALGORİTMASI

Bölüm III-E’deki sigmoid ve tanh arama tabloları Python’da çevrim dışı olarak önceden hesaplanır ve C başlık dosyası olarak yayılır:

```
LUT_SIZE      = 256
INPUT_MIN     = -8.0
INPUT_MAX     = 8.0
BUCKET_WIDTH  = (INPUT_MAX - INPUT_MIN) / LUT_SIZE

sigmoid_lut = [
    1.0 / (1.0 + math.exp(-(INPUT_MIN
        + (i + 0.5) * BUCKET_WIDTH)))
    for i in range(LUT_SIZE)
]
tanh_lut = [
    math.tanh(INPUT_MIN + (i + 0.5) * BUCKET_WIDTH)
    for i in range(LUT_SIZE)
]

Her giriş, kendi kovasının merkezindeki aktivasyon değerini
saklar ((i + 0.5) ofseti). Bu, kovacık içinde düzgün dağılmış
bir giriş için maksimum olabilirlik tahminidir ve yarım kova
yanlılığını önler. Çalışma zamanı araması şöyledir:

static inline float lut_eval(const float *lut, float x) {
    if (x <= LUT_INPUT_MIN) return lut[0];
    if (x >= LUT_INPUT_MAX) return lut[LUT_SIZE - 1];
    int idx = (int)((x - LUT_INPUT_MIN) * LUT_INPUT_SCALE);
    return lut[idx];
}
```

LUT_INPUT_SCALE = 1 / BUCKET_WIDTH olarak tanımlanır. İki tablo birlikte 2 KB Flash kaplar; birlikte çalışma zamanındaki tüm expf ve tanhf çağrılarını ortadan kaldırır.

EK D
MSP430 I²C İLK ÇALIŞTIRMA

MSP430G2553 üzerinde canlı MPU-6050 okuması, standart pin atamasıyla (P1.6 = SCL, P1.7 = SDA) I²C ana kipindeki USCI_B0 çevre birimini kullanır. Gelecekteki okurlar için iki pratik hususu kaydetmeye değer:

- **J5 jumper'ının çıkarılması.** MSP-EXP430G2 Launch-Pad, kart üzerindeki yeşil LED'i P1.6 ile çoklar. Herhangi bir I²C iletişiminden önce J5 jumper'ı fiziksel olarak çıkarılmalıdır; aksi hâlde SCL LED tarafından yüklenir ve veri yolu I²C yüksek seviye eşiğinin altında kalır.
- **Bus-busy kurtarma.** Aktarım ortasında güç kesilirse USCI_B0 çevre birimi UCSCLLOW + UCBBUSY durumuna kilitlenebilir. Geleneksel 9-saat kurtarma dizisi (SDA serbestken SCL'yi dokuz kez değiştirip ardından STOP üretmek) veri yolunu geri yükler. Açılıştaki GPIO düzeyinde yapılan bir sağlık denetimi (P1IN & (BIT6|BIT7) değerinin 0xC0 olması beklenir), I²C durum makinesi devreye alınmadan önce bu koşulu sıptar.

Bu iki madde yaklaşık bir günlük ilk çalıştırma süresi tüketmiştir ve standart MSP430 uygulama notlarında [27] belgelenmemiştir. Bir sonraki okura aynı günü kazandırma olasılığı için buraya eklenmiştir.